Software Reengineering

Model answers for the paper: short English definitions and explanations

1.1 Legacy system kya hota hai?

A legacy system is an old software system built with outdated technology that is still used because it performs critical business functions. It is usually hard to understand, modify and maintain. Example: an old bank system written in COBOL running on a mainframe.

1.2 Legacy systems ki khasiyatein (pehchan)

Characteristics of legacy systems: old and large, business-critical, poor or missing documentation, original developers have left, outdated technology, difficult and costly to change, and tangled (spaghetti) code structure.

1.3 Legacy system phir bhi kyun chalta rehta hai?

Legacy systems stay in use because replacing them is costly and risky, they contain valuable business rules built up over many years, and the business depends on them every day.

1.4 Legacy systems ke masle

Problems of legacy systems: high maintenance cost, missing documentation, shortage of skilled staff, difficult integration with new systems, obsolete hardware, weak security, and gradual decay of the software structure.

1.5 Legacy system ke hisse

A legacy system consists of hardware, support software, application software, application data, business processes and business policies and rules.

1.6 Legacy system ka kya kiya jaye? (4 raaste)

There are four options for a legacy system: scrap it, keep maintaining it, reengineer it, or replace it with a new system. If the system has high business value but poor technical quality, reengineering is the best option.

2.1 Software Reengineering kya hai?

Software reengineering is the examination and alteration of an existing system to reconstitute it in a new form, and the implementation of that new form. It improves quality and maintainability while keeping the same functionality. It includes reverse engineering, restructuring and forward engineering.

2.2 Reengineering kyun karte hain?

Reasons for reengineering: lower cost and risk than rewriting, preserving existing business knowledge, better maintainability, moving to new technology, and recovering lost documentation.

2.3 Maintenance, Reengineering aur Naya system mein farq

Maintenance makes small changes after delivery, reengineering restructures a large part of the system without changing its function, and replacement builds a completely new system. Cost and risk are lowest for maintenance and highest for replacement.

2.4 Zaroori Terms: Forward, Reverse, Restructuring waghera

Forward engineering moves from design to code. Reverse engineering analyzes existing code to recover its design and specification, without changing it. Restructuring transforms the structure of a system at the same abstraction level without changing its behavior. Reengineering combines reverse engineering, restructuring and forward engineering.

2.5 Reengineering ki 2 strategies

In the big-bang strategy the whole system is reengineered at once, which is fast but risky. In the incremental strategy the system is reengineered part by part, which is safer because errors are found early, but needs extra effort to connect old and new parts.

2.6 Reengineering ke fayde aur nuqsan

Advantages: cheaper and less risky than new development, keeps business knowledge, improves quality. Disadvantages: needs time and effort, and cannot fix every limit of a very poor design.

3.1 Software Maintenance kya hai?

Software maintenance is the modification of a software product after delivery to correct faults, improve performance or other attributes, or adapt it to a changed environment.

3.2 Maintenance ki 4 qismein

Types of maintenance: Corrective fixes faults. Adaptive changes the software for a new environment such as a new OS or law. Perfective adds features or improves performance. Preventive improves the structure to avoid future problems.

3.3 Software Evolution kya hai?

Software evolution is the continuous process of changing and growing a software system over its life to meet changing needs and technology.

3.4 Lehman ke 8 Laws (aasan misalon ke saath)

Lehman's laws describe how software changes over time: continuing change, increasing complexity, self-regulation, conservation of organisational stability, conservation of familiarity, continuing growth, declining quality and feedback system. For example, under the law of continuing change, software that is not updated becomes less useful.

3.5 Maintenance mehnga kyun hota hai?

Maintenance is expensive because of team instability, lack of staff experience, old and complex programs, poor documentation, and the ripple effect, where a change in one place causes errors elsewhere.

3.6 Maintenance process (qadam ba qadam)

The maintenance process: change request, impact analysis, planning and design, implementation, regression testing, and release with updated documentation.

3.7 Software ki zindagi ke stages

Software life stages: initial development, evolution, servicing, phase-out and close-down. Reengineering can bring a system in the servicing stage back to the evolution stage.

Qadam 1: Inventory Analysis (list banao)

Inventory analysis: prepare a list of all existing applications with details such as age, size, business importance and maintenance cost, and then select the best candidates for reengineering.

Qadam 2: Document Restructuring (documentation theek karo)

Document restructuring deals with weak documentation. The options are: do nothing if the system is stable, document only the parts that change, or fully redocument a business-critical system.

Qadam 3: Reverse Engineering (samjho)

Reverse engineering analyzes the existing program to recover its design, data structures and functionality. Nothing is changed; the aim is to understand the system.

Qadam 4: Code Restructuring (code saaf karo)

Code restructuring analyzes the source code and rewrites unstructured parts into a clean, well-structured form without changing behavior. The result is reviewed and tested.

Qadam 5: Data Restructuring (data saaf karo)

Data restructuring cleans and reorganizes the data architecture. Existing data structures are analyzed, then data models, names and formats are standardized, for example by moving old flat files into a well-designed database.

Qadam 6: Forward Engineering (naya banao)

Forward engineering uses the information gained from reverse engineering to build a new and better system, often on a new platform, reusing the valuable business logic of the old one.

Topic 1: Software Restructuring

Software restructuring is the improvement of a system's internal structure without changing its external behavior. It can be applied to code, data and architecture. Its purpose is better understandability, easier maintenance and fewer defects.

Topic 2: Restructuring ki Definition

Restructuring is the transformation of a system from one representation to another at the same abstraction level, while preserving its external behavior (Chikofsky and Cross).

Topic 3: Code Restructuring ki Definition

Code restructuring is modifying the source code to improve its structure and readability without changing its external behavior. It typically converts unstructured, spaghetti code into clean, modular code by removing GOTOs, deep nesting and dead code.

Topic 4: Refactoring

Refactoring is a disciplined technique of restructuring existing code by making small changes that improve its internal structure without changing its external behavior (Fowler). It is done in small steps with tests after each step. Common techniques are Extract Method, Extract Class, Rename and Move Method.

Topic 5: Code Smell

A code smell is a surface symptom in the source code that usually indicates a deeper design problem. It is not a bug, but a sign that the code should be refactored. Common smells are Long Method, Large Class, Duplicate Code, Long Parameter List, Feature Envy and Shotgun Surgery.

Topic 6: Function Extraction (Extract Method)

Function extraction (Extract Method) means moving a fragment of code from a long function into a new function with a meaningful name and replacing the fragment with a call to it. It removes the Long Method smell, improves readability, avoids duplication and makes testing easier.

Topic 7: Class Extraction (Extract Class)

Class extraction (Extract Class) means splitting a large class by moving related fields and methods into a new class. It follows the Single Responsibility principle, so each class does one job. It makes classes smaller, easier to understand and test, and allows reuse.

Topic 8: Duplicate Code

Duplicate code is the same or very similar code repeated in several places, usually through copy-paste. It increases maintenance cost and the risk of inconsistent bug fixes. It is removed with refactoring such as Extract Method, Pull Up Method and Template Method. Clones are classified as Type 1 to Type 4.

Topic 9: Code Reliability Improved

Code reliability is improved by restructuring and refactoring because the code becomes cleaner, smaller and less complex. Removing duplicate and dead code, reducing complexity, adding error handling and unit tests, and lowering coupling all reduce defects. Reliability can be measured with defect density and mean time between failures (MTBF).